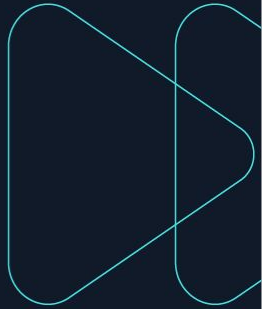


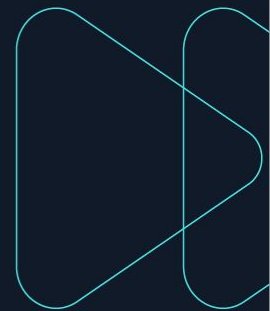
MACHINE LEARNING CLASIFICACIÓN Y REGRESIÓN



Métodos de validación de información

Módulo 2

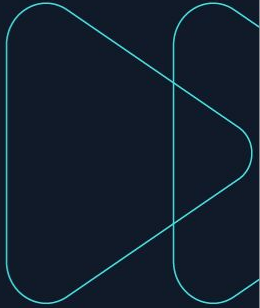
Unidad 1: Evaluación de modelos



Unidad 2

- ¿Cómo evaluar bien un modelo?
- Split de datos
- Validación cruzada
- Optimización de hiperparámetros

¿Cómo evaluar?



¿Como evaluar correctamente un modelo?

En Machine Learning no basta con entrenar un modelo. El objetivo principal es que el modelo **generalice correctamente**, es decir, que pueda realizar buenas predicciones sobre datos nuevos que nunca vio durante el entrenamiento.

Por esta razón, uno de los problemas centrales del aprendizaje automático es cómo estimar correctamente el desempeño de un modelo. Un modelo puede parecer muy preciso durante el entrenamiento, pero fallar cuando se aplica en producción.



Estimar el error de generalización

Medir cómo se comportará el modelo con datos nuevos



Comparar distintos modelos

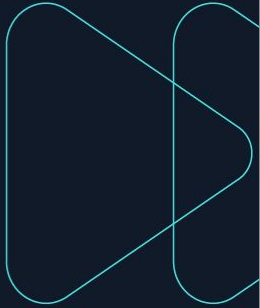
Elegir la mejor configuración entre múltiples opciones

Para lograr esto se utilizan estrategias como división de datos, validación cruzada y optimización de hiperparámetros.



Ejemplo: Si entrenamos el modelo con 10.000 clientes y lo evaluamos sobre esos mismos 10.000, el modelo podría parecer excelente. Pero cuando se aplica a clientes nuevos, el desempeño puede caer drásticamente. Esto ocurre por **overfitting**.

Split de datos



Split de datos

El método más simple para evaluar un modelo consiste en dividir el dataset en dos subconjuntos: Entrenamiento (train) y Prueba (test).

El modelo se entrena con el conjunto de entrenamiento y luego se evalúa utilizando el conjunto de prueba. La idea fundamental es que el conjunto de test funcione como una simulación de datos futuros, permitiendo estimar cómo se comportará el modelo en situaciones reales.

Ejemplo: Predicción de fraude bancario

Dataset

- 500.000 transacciones
- Variable objetivo: fraude / no fraude

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

División

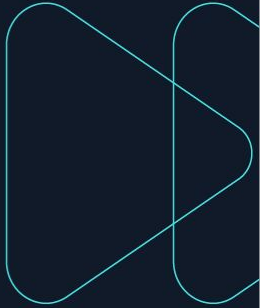
Train: 400.000

Test: 100.000

El modelo nunca ve el test durante el entrenamiento

Problema: El resultado puede depender mucho de cómo se hizo el split. Dos splits diferentes del mismo dataset pueden producir evaluaciones bastante distintas.

Validación cruzada



Validación cruzada (CV)

Permite obtener una **estimación más robusta del desempeño de un modelo** utilizando múltiples particiones del dataset.

Consiste en repetir el proceso de entrenamiento y evaluación varias veces con diferentes particiones del dataset. De esta manera, cada observación del dataset se utiliza tanto para entrenamiento como para evaluación, pero en iteraciones diferentes.

Esto reduce la dependencia de una única división arbitraria de los datos.

Validación cruzada (CV)

K-Fold

Es el método de validación cruzada más utilizado. El dataset se divide en **K partes iguales** llamadas folds.

01

Dividir el dataset en K subconjuntos

02

Repetir K veces

En cada iteración: usar K-1 folds para entrenar y 1 fold para validar

03

Calcular el promedio

Obtener K métricas de desempeño y calcular su promedio

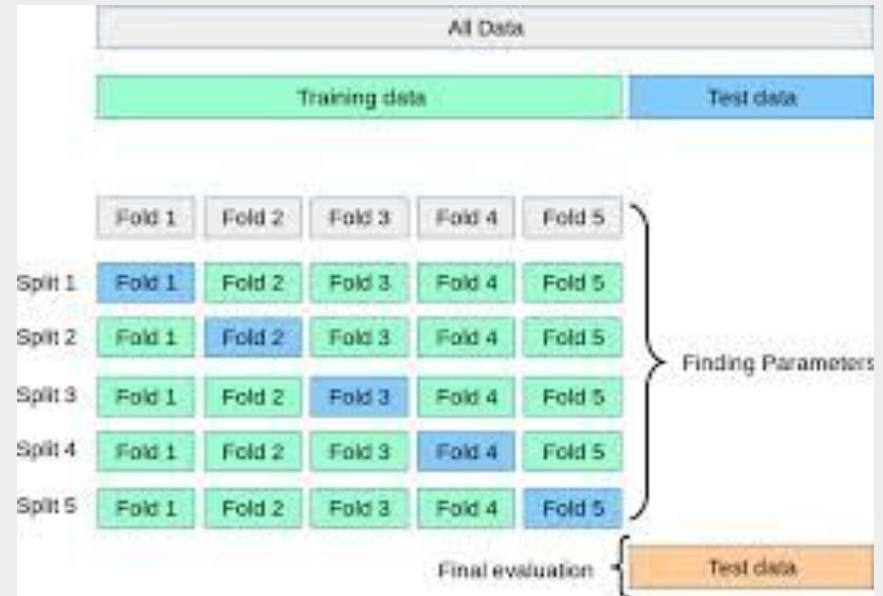
Validación cruzada (CV)

Cómo implementar en scikit-learn:

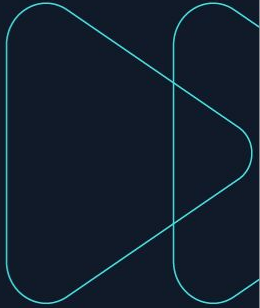
```
from sklearn.model_selection import  
cross_val_score, KFold
```

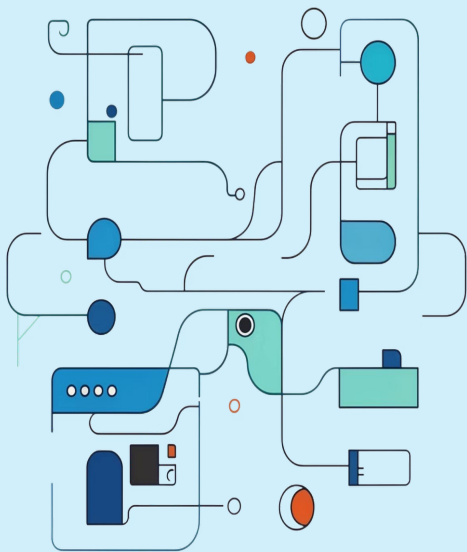
```
clf = LogisticRegression()
```

```
kf = KFold(n_splits=5,  
shuffle=True, random_state=42)  
scores = cross_val_score(clf, X, y,  
cv=kf, scoring='accuracy')
```



Hiperparámetros





Hiperparámetros

En machine learning existen dos tipos de parámetros que son fundamentales para el funcionamiento de los modelos.

Parámetros

Se **aprenden automáticamente** durante el entrenamiento.

Ejemplo en regresión lineal:

- Coeficientes, Intercepto

Hiperparámetros

Son configuraciones que **define el usuario antes de entrenar el modelo**. **influyen fuertemente en el rendimiento del modelo** y deben ser cuidadosamente seleccionados

Ejemplos:

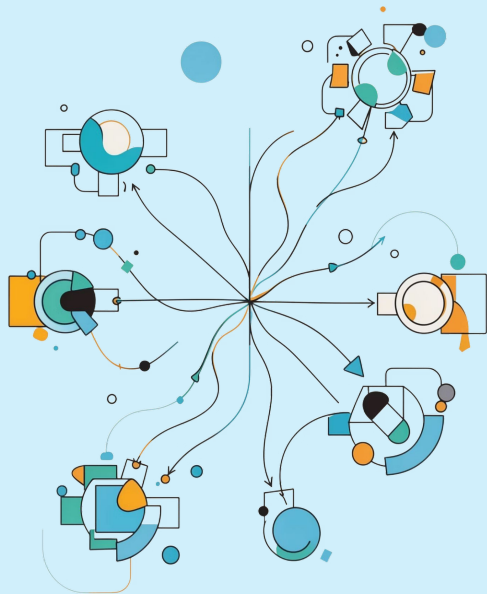
Árbol de decisión: max_depth, min_samples_split

Random Forest: n_estimators, max_features

SVM: C, gamma

Optimización

Consiste en encontrar la combinación de hiperparámetros que **maximiza el rendimiento del modelo**.



Definir espacio de búsqueda

Establecer el rango de valores posibles para cada hiperparámetro



Entrenar múltiples modelos

Probar diferentes configuraciones con validación cruzada



Evaluar cada configuración

Calcular métricas de desempeño para cada combinación



Seleccionar la mejor

Elegir la combinación con el mejor resultado

Métodos de optimización

Grid Search

Prueba **todas las combinaciones posibles** de hiperparámetros.

Ejemplo: max_depth = [3,5,7],
min_samples_split = [2,10]

Total: 6 modelos

Problema: El número de combinaciones puede crecer exponencialmente

Random Search

No prueba todas las combinaciones.

Selecciona combinaciones aleatorias del espacio de hiperparámetros.

Ventaja: Explora más combinaciones diferentes en menos tiempo. Muchos hiperparámetros no afectan demasiado el rendimiento.

Optimización Bayesiana

Enfoque más sofisticado. No probar hiperparámetros al azar, sino **aprender qué configuraciones parecen prometedoras.**

Ventajas: Mucho más eficiente en problemas grandes, requiere menos evaluaciones.

Analogía intuitiva

Buscar el mejor restaurante en una ciudad:

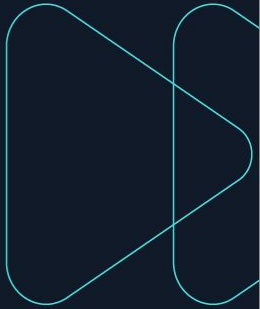
Grid search: probar todos

Random search: probar al azar

Optimización bayesiana: usar recomendaciones basadas en experiencias previas

Muy usada en deep learning, AutoML y tuning de modelos complejos.

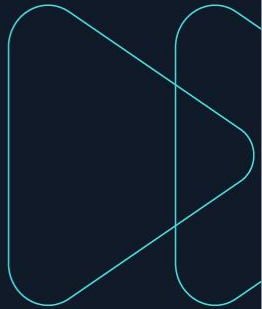
Resumen



Resumen de la clase

- Comprender por qué no se debe evaluar un modelo con los mismos datos con los que se entrena.
- Aplicar correctamente split de datos.
- Comprender cómo funciona la validación cruzada (K-Fold).
- Entender qué son los hiperparámetros y por qué deben optimizarse.
- Conocer métodos comunes de optimización:
 - Grid Search
 - Random Search
 - Optimización Bayesiana.

¿Consultas?



¡Muchas gracias!

